

11. Ziad.

Contents

Two Sat	2
Min Cost Max Flow	3
SoS DP	4

Two Sat

```
1 struct two_sat {
2     int n, timer, scc_cnt;
3     vector<vector<int> > g;
4     vector<int> dfn, low, scc, st, answer;
5     vector<char> in_st;
6
7     two_sat(int n) : n(n), timer(0), scc_cnt(0) {
8         g.assign(2 * n, {});
9         dfn.assign(2 * n, 0);
10        low.assign(2 * n, 0);
11        scc.assign(2 * n, 0);
12        in_st.assign(2 * n, false);
13        answer.assign(n, 0);
14    }
15
16    int var(int i, bool val) { return i + (val ? 0 : n); }
17
18    void add_edge(int u, int v) { g[u].push_back(v); }
19
20    void add_imply(int i, bool f, int j, bool k) {
21        add_edge(var(i, f), var(j, k));
22    }
23
24    void set_true(int i) { add_imply(i, false, i, true); }
25    void set_false(int i) { add_imply(i, true, i, false); }
26
27    void add_or(int i, bool f, int j, bool k) {
28        add_imply(i, !f, j, k);
29        add_imply(j, !k, i, f);
30    }
31
32    void add_xor(int i, bool f, int j, bool k) {
33        add_or(i, f, j, k);
34        add_or(i, !f, j, !k);
35    }
36
37    void add_xnor(int i, bool f, int j, bool k) {
38        add_or(i, f, j, !k);
39        add_or(i, !f, j, k);
40    }
41
42    void add_and(int i, bool f, int j, bool k) {
43        add_imply(i, !f, i, f);
44        add_imply(j, !k, j, k);
45    }
46
47    void dfs(int u) {
48        dfn[u] = low[u] = ++timer;
49        st.push_back(u);
50        in_st[u] = true;
51        for (int v: g[u]) {
52            if (!dfn[v]) {
53                dfs(v);
54                low[u] = min(low[u], low[v]);
```

```

55         } else if (in_st[v]) {
56             low[u] = min(low[u], dfn[v]);
57         }
58     }
59     if (low[u] == dfn[u]) {
60         scc_cnt++;
61         while (true) {
62             int v = st.back();
63             st.pop_back();
64             in_st[v] = false;
65             scc[v] = scc_cnt;
66             if (u == v) break;
67         }
68     }
69 }
70
71 bool satisfiable() {
72     timer = scc_cnt = 0;
73     fill(dfn.begin(), dfn.end(), 0);
74
75     for (int i = 0; i < 2 * n; i++)
76         if (!dfn[i]) dfs(i);
77
78     for (int i = 0; i < n; i++) {
79         if (scc[i] == scc[i + n]) return false;
80         answer[i] = scc[i] < scc[i + n];
81     }
82     // answer[i] = 0 ==> negative
83     return true;
84 }
85 };

```

Min Cost Max Flow

```

1  const int inf = 1e9;
2  class MinCostMaxFlow {
3      struct Edge { int to, rev, cap, cost; };
4      int n;
5      vector<vector<Edge>> g;
6      vector<int> dis, prv, pri, h;
7
8  public:
9      MinCostMaxFlow(int n) : n(n), g(n), dis(n), prv(n), pri(n), h(n) {}
10
11     void addEdge(int u, int v, int cap, int cost) {
12         g[u].push_back({v, (int)g[v].size(), cap, cost});
13         g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
14     }
15
16     pair<int, int> flow(int s, int t, int maxF = inf) {
17         int res = 0, flow = 0;
18         fill(h.begin(), h.end(), 0);
19
20         while (maxF > 0) {
21             priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> q;
22             fill(dis.begin(), dis.end(), inf);

```

```

23         dis[s] = 0, q.emplace(0, s);
24
25         while (!q.empty()) {
26             auto [d, v] = q.top(); q.pop();
27             if (dis[v] < d) continue;
28             for (int i = 0; i < g[v].size(); i++) {
29                 auto& e = g[v][i];
30                 if (e.cap > 0 && dis[e.to] > dis[v] + e.cost + h[v] - h[e.to]) {
31                     dis[e.to] = dis[v] + e.cost + h[v] - h[e.to];
32                     prv[e.to] = v, pri[e.to] = i;
33                     q.emplace(dis[e.to], e.to);
34                 }
35             }
36         }
37         if(dis[t] == inf) break;
38
39         for(int v = 0; v < n; ++v) h[v] += dis[v];
40         int d = maxF;
41         for(int v = t; v != s; v = prv[v]) d = min(d, g[prv[v]][pri[v]].cap);
42
43         maxF -= d, flow += d, res += d * h[t];
44         for (int v = t; v != s; v = prv[v]) {
45             auto& e = g[prv[v]][pri[v]];
46             e.cap -= d, g[v][e.rev].cap += d;
47         }
48     }
49     return {flow, res};
50 }
51 };

```

SoS DP

```

1  Computes sum over all subsets (or superset) of bitmask values efficiently using bitwise DP.
2
3  f[x]++;
4
5  f1[(N - 1) & ~x]++;
6
7  number of y such that:
8
9  x | y = x => f[x]
10
11 x & y = x => f1[(N - 1) & ~x]
12
13 x & y = 0 => f[(N - 1) & ~x]
14
15 !invert -> sum of subsets, invert -> undo pre operation
16 void sos(vector<int> &dp, bool invert = false) {
17     for (int i = 0; 1 << i < dp.size(); i++) {
18         for (int mask = 0; mask < dp.size(); mask++) {
19             if (mask & (1 << i)) {
20                 dp[mask] += invert ? -dp[mask ^ (1 << i)] : dp[mask ^ (1 << i)];
21                 // dp[mask] >= mod? dp[mask] -= mod: dp[mask] < 0? dp[mask] += mod: 0;
22             }
23         }
24     }

```

```

25 }
26
27 template<typename T>
28 void SubsetZetaTransform(vector<T>& v) {
29     const int n = v.size();
30     for (int j = 1; j < n; j <= 1) {
31         for (int i = 0; i < n; i++)
32             if (i & j)
33                 v[i] += v[i ^ j];
34     }
35 }
36
37 template<typename T>
38 void SubsetMobiusTransform(vector<T>& v) {
39     const int n = v.size();
40     for (int j = 1; j < n; j <= 1) {
41         for (int i = 0; i < n; i++)
42             if (i & j)
43                 v[i] -= v[i ^ j];
44     }
45 }
46
47 template<typename T>
48 void SupersetZetaTransform(vector<T>& v) {
49     const int n = v.size(); // n must be a power of 2
50     for (int j = 1; j < n; j <= 1) {
51         for (int i = 0; i < n; i++)
52             if (i & j)
53                 v[i ^ j] += v[i];
54     }
55 }
56
57 template<typename T>
58 void SupersetMobiusTransform(vector<T>& v) {
59     const int n = v.size(); // n must be a power of 2
60     for (int j = 1; j < n; j <= 1) {
61         for (int i = 0; i < n; i++)
62             if (i & j)
63                 v[i ^ j] -= v[i];
64     }
65 }
66
67 1. Count how many array elements are submasks of x.
68     freq[value]++
69     SubsetZetaTransform(freq);
70     Then:
71         freq[x]
72     = number of input values y such that
73         y ⊆ x
74
75
76 2. Count how many array elements are supermasks of x.
77     freq[value]++
78     SupersetZetaTransform(freq);
79     Then:
80         freq[x]
81     = number of input values y such that

```

```

82         y  $\supseteq$  x
83
84 3. Recover the original array after a transform.
85     SubsetZetaTransform(freq);
86     SubsetMobiusTransform(freq);
87     // freq is back to its original values
88
89 4. Disjoint masks.
90     x & y == 0
91     y must be a submask of ~x.
92     For B bits:
93         complement = (1 << B) - 1 ^ x
94     So after subset-zeta preprocessing:
95         count = freq[complement]

```